


```

int i=0;          3
do{              4
    x[i]+=2;      5
    i+=2;         6
}                7
while ( i<=n );   8
i=1;             9
while ( i<=(n/2)) 10
{               11
    x[i]+=x[i+1]; 12
    i++;          13
}               14
}              15

```

Line	s/e	frequency	total step
1	0	1	0
2	0	1	0
3	1	1	1
4	0	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)	0
5	1	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)
6	1	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)
7	0	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)	0
8	0	(n/2)+2(若 n 为偶数) (n+1)/3(若 n 为奇数)	0
9	1	1	1
10	0	(n/2)+1(若 n 为偶数) (n+1)/2(若 n 为奇数)	0
11	0	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)	0
12	1	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)
13	1	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)
14	0	n/2(若 n 为偶数) (n-1)/2(若 n 为奇数)	0
15	0	1	0

So

for $n \% 2 == 0$, $t_D(n) = 2n + 4$,

for $n \% 2 != 0$, $t_D(n) = 2n + 2$

5.

(a)

```
void Transpose (int ** a, int n)
{
    int count=0;
    for(int i = 0; i < n - 1; i++)
        for(int j = i + 1; j < n; j++)
        {
            swap ( a[i][j],a[j][i] );
            count++;
        }
}
```

(c)count 为 $n(n-1)/2$

(d)

```
void Transpose (int ** a, int n)          1
{                                          2
    for(int i = 0; i < n - 1; i++)        3
        for(int j = i + 1; j < n; j++)    4
            swap ( a[i][j],a[j][i] );    5
                                          6
}
```

Line	s/e	frequency	total step
1	0	1	0
2	0	1	0
3	1	$n-1$	$n-1$
4	1	$n(n-1)/2$	$n(n-1)/2$
5	1	$n(n-1)/2$	$n(n-1)/2$
6	0	1	0

So

$t_{\text{Transpose}}(n)=n^2-1$

6.

(a)

```
void Multiply (int ** a,int ** b, int ** c, int n)
{
    int count=0;
    for( int i = 0; i < n; i++)
    {
        c[i][j] = 0;
        for( int k = 0; k < n; k++)
        {
            c[i][j] += a[i][k] * b[k][i];
            count++;
        }
    }
}
```

2

```

for( int i = 0; i < m; i++)           3
    for( int j=0; j < p; j++)         4
    {                                 5
        c[i][j] = 0;                 6
        for( int k = 0; k < n; k++)   7
            c[i][j] += a[i][k] * b[k][i]; 8
    }                                 9
}                                     10

```

Line	s/e	frequency	total step
1	0	1	0
2	0	1	0
3	1	m	m
4	1	m*p	m*p
5	0	m*p	0
6	1	m*p	m*p
7	1	m*p*n	m*p*n
8	1	m*p*n	m*p*n
9	0	m*p	0
10	0	1	0

So

$$t_{\text{Multiply}}(m,n,p)=m+2m*p+2m*p*n$$

(b)

如果 **b** 矩阵的列比 **a** 矩阵的行在内存中更连续，那么交换最外层 **for** 循环的顺序可能会对减少运算时间有利。